

A Composable AI-Accelerated Iterative Solver for 3D-IC Thermal Modeling

Yixing Li, Jiahang Zhou, Zhiyu Zeng, Xin Ai

Abstract—Accurate thermal analysis of heterogeneous 2.5D/3D-IC packages is essential yet computationally prohibitive. A single full-package FEM simulation can take hours, while AI-based surrogates treat the entire stack as a monolithic prediction target and must be retrained whenever the die count or topology changes. To address this limitation, this work proposes Domain-Decomposed AI-Accelerated Iterative Solver for Thermal Analysis (DAIST), a composable thermal solver that decomposes the global package simulation into block-level subdomain problems, replaces subdomain solvers with neural operators, and couples them through iterative exchanges of interfacial temperature and heat flux. This local-to-global architecture eliminates the topology lock-in of monolithic models: block-level neural operators can be directly reused in unseen package assemblies without retraining. The iterative coupling strategy further provides a controllable accuracy–runtime tradeoff, where the iteration budget can be adjusted to trade accuracy for runtime. Evaluated on a multi-chiplet system and an advanced packaging system, DAIST achieves up to $178\times$ speedup over traditional FEM solvers with mean temperature errors of 0.068% and 0.323%, respectively, while demonstrating cross-topology reuse of block-level models across structurally distinct package assemblies.

Index Terms—3D-IC, thermal simulation, domain decomposition, operator learning

I. INTRODUCTION

Thermal analysis has become increasingly critical in modern heterogeneous 2.5D and 3D integrated packages. Unlike traditional monolithic single-die designs, contemporary package architectures consist of multiple thermally coupled components such as logic dies, high-bandwidth memory (HBM) stacks, interposers, and interface materials [1], [2]. Heat generated in one component can propagate across neighboring blocks through physical contact regions, creating complex thermal interactions throughout the package. Accurate evaluation of such multi-block thermal systems is essential for ensuring reliability, preventing localized thermal hot spots, and enabling thermal-aware design decisions.

High-fidelity numerical solvers such as the finite-element method (FEM) and finite-volume method (FVM) are widely used for package-level thermal analysis in tools such as Ansys IcePak, Cadence Celsius, and COMSOL [3]–[8]. While these methods provide accurate solutions to the governing heat equations, their computational cost grows rapidly with mesh resolution, material heterogeneity, and system size. For

complex multi-die packages, a single full-package simulation can take minutes to hours, making repeated thermal evaluation impractical during iterative design workflows.

With the rapid evolution of machine learning, data-driven and physics-informed approaches have attracted growing attention for thermal modeling. Physics-informed neural networks (PINNs) [9] incorporate governing equations into the loss function to improve physical consistency, but remain tied to fixed design configurations and typically require retraining for each design. Operator-learning approaches instead learn mappings between function spaces and have shown promising performance for partial differential equation (PDE) surrogate modeling. For example, DeepOHeat [10] and DeepOHeat-v1 [11] map power and boundary-condition inputs to 3D temperature fields with significant speedups over FEM, while PI-ONet [12] extends this approach to multilayer chiplet stacks. Neural operators such as the Fourier Neural Operator (FNO) [13] and the U-shaped Neural Operator (U-NO) [14] further improve data efficiency and scalability for PDE-based surrogate modeling, enabling significant acceleration of thermal simulations [15].

While effective for fixed package configurations, monolithic models have several limitations for heterogeneous package architectures. First, the learned model is strongly tied to the entire package topology, making it difficult to scale trained models across different numbers of dies, memory stacks, and handle changes in interface regions. Second, interface physics between neighboring components is implicitly embedded within the model rather than explicitly enforced. As a result, temperature continuity and heat-flux balance across interfaces cannot be monitored or controlled during inference, limiting the model’s physical interpretability.

These limitations are particularly problematic for heterogeneous packages, which naturally consist of multiple physical components with different materials, geometries, and thermal behaviors. Instead of learning a single global predictor, it is desirable to treat each component as an independent thermal block whose behavior can be modeled separately and flexibly composed into larger package systems.

Domain decomposition methods (DDM) provide a natural framework for addressing such coupled multi-domain problems. In classical DDM, a large computational domain is partitioned into smaller subdomains, each solved locally while exchanging boundary information with neighboring subdomains. Iterative updates of interface boundary conditions ensure that the local solutions converge to a globally consistent solution

Yixing Li, Zhiyu Zeng, and Xin Ai are with Cadence Design Systems (email: yixingli@cadence.com; zzeng@cadence.com; nathanai@cadence.com). Jiahang Zhou is with the University of Notre Dame (email: jzhou8@nd.edu).

[16]. Recently, several studies have explored combining neural operators with domain decomposition to accelerate the solution of PDEs [17], [18]. These approaches demonstrate the potential of learned models as local subdomain solvers within an iterative framework.

However, existing learning-based DDM approaches are primarily developed for canonical PDE settings, where the global domain is partitioned into reusable local subproblems or congruent subdomains. Their main focus is geometry generalization and efficient local surrogate reuse, rather than modular thermal assembly of heterogeneous blocks. In contrast, 3D-IC packages are inherently compositional thermal systems consisting of different blocks. These blocks may exhibit different materials, internal structures, and modeling requirements while being strongly coupled through shared interfaces. Modeling such systems therefore requires explicit handling of interface coupling as well as the ability to combine different solver types within a unified framework.

In this work, we adopt a compositional perspective and formulate heterogeneous package thermal analysis as a modular multi-block system. Each physical component is represented by an independent block-level thermal predictor, while global thermal consistency is enforced through iterative exchange of interface boundary conditions among adjacent blocks. This formulation allows flexible composition of heterogeneous blocks when simulating larger package assemblies, and supports reusable component-level models.

The main contributions of this work are summarized as follows:

- We formulate heterogeneous multi-block thermal analysis as a compositional thermal modeling framework, where each physical component (e.g., dies, HBMs, interposers) is represented by an independent block-level thermal predictor connected through physical interfaces. This formulation enables flexible composition of heterogeneous blocks into larger package assemblies, avoiding the tight coupling inherent in monolithic AI models that learn the entire package as a single predictor.
- We propose an interface-iterative thermal solving scheme that explicitly exchanges temperature and heat-flux boundary conditions between neighboring blocks. The iterative formulation enables controllable trade-offs between solution accuracy and runtime by adjusting the iteration budget, while enforcing temperature continuity and heat-flux balance across interfaces.

II. METHODOLOGY

A. Thermal Problem Formulation

This work considers steady-state heat conduction in heterogeneous multi-block systems. Let Ω denote the global thermal domain composed of multiple physically connected blocks. The temperature distribution $T(\mathbf{x})$ satisfies the heat conduction equation,

$$\nabla \cdot (k \nabla T) + Q = 0, \quad (1)$$

where k denotes the thermal conductivity and Q represents the internal heat generation. The global domain Ω is partitioned into N non-overlapping thermal blocks, as described in Eq. 2.

$$\Omega = \bigcup_{b=1}^N \Omega_b, \quad \Omega_i \cap \Omega_j = \emptyset \quad (i \neq j) \quad (2)$$

Each block in Eq. 2 corresponds to a physical component in the package, such as a logic die, HBM stack, interposer, etc. Therefore, the decomposition follows the physical assembly structure of heterogeneous packages rather than using any artificial computational partitions.

Standard thermal boundary conditions are applied to external boundaries, including Dirichlet (fixed temperature), Neumann (prescribed heat flux), and Robin (convective cooling) conditions. Blocks interact through shared physical interfaces. For each interface Γ_{ij} between neighboring blocks Ω_i and Ω_j , the physical solution must satisfy

- temperature continuity: $T_i = T_j$ on Γ_{ij} ,
- heat-flux balance: $k_i \nabla T_i \cdot \mathbf{n}_i = -k_j \nabla T_j \cdot \mathbf{n}_j$ on Γ_{ij} ,

which ensure global thermal consistency across block boundaries.

B. Block Decomposition and Interface Coupling

To enable modular thermal solving, the global domain is decomposed into individual blocks that are solved locally. Each block receives boundary information from its neighboring blocks through shared interfaces. For each interface Γ_{ij} between two blocks, the temperature and heat flux are exchanged to serve as updated boundary conditions for the next iteration. At iteration k , each block receives interface boundary conditions computed from its neighbors during iteration $k-1$. To be specific, in the proposed coupling scheme, temperature at the interface is passed from one block to its neighbor as a Dirichlet boundary condition, while the receiving block returns the corresponding heat flux at the same interface.

C. Interface-Iterative Thermal Solver

The global thermal solution is obtained through a two-phase iterative procedure illustrated in Figure 1.

a) *Phase 1 (interface iteration)*: At each iteration k , every block solves a *boundary-level* subproblem with boundary conditions obtained from the last-iteration interface quantities or initial-step conditions. Given the interface conditions from neighboring blocks, the local solver returns updated interface temperature and heat flux at shared surfaces. These interface quantities are exchanged between neighboring blocks and smoothed via relaxation:

$$u^{(k)} = u^{(k-1)} + \omega_u \left(u_{\text{neighbor}}^{(k-1)} - u^{(k-1)} \right), \quad (3)$$

where $\omega_u \in (0, 1]$ is a variable-specific relaxation parameter whose role is formalized in Section II-F. The interface iteration repeats until the mean temperature change across all interfaces satisfies

$$\delta^{(k)} < \tau, \quad (4)$$

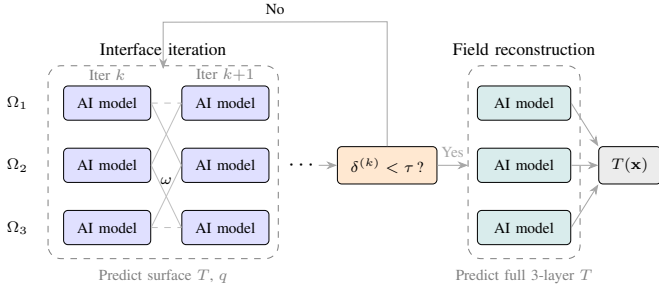


Fig. 1: Overview of the DAIST two-phase algorithm. **Interface iteration** (left): each block’s AI model predicts surface temperature and heat flux, and interface quantities are exchanged between neighboring blocks with relaxation parameter ω . The process repeats until convergence ($\delta^{(k)} < \tau$). **Field reconstruction** (right): a single forward pass per block produces the full three-layer temperature field, which is assembled into the global solution field $T(\mathbf{x})$.

where τ is a user-specified tolerance. In this phase, only surface quantities are predicted, so each local solve is lightweight.

b) Phase 2 (field reconstruction): Once interface convergence is achieved, a single *volume-level* forward pass is executed: each block solver takes the converged interface boundary conditions and produces the full three-layer temperature field (substrate, device, interconnect) within that block. The per-block fields are then assembled into the global temperature solution.

The two-phase design keeps the iterative loop inexpensive by restricting repeated evaluation to interface quantities, while the one-shot field reconstruction amortizes the cost of generating the full volumetric output.

D. Local Block Solvers

Within the proposed framework, each block is solved independently using a local solver, which is an AI-based surrogate model in this work.

The local solver takes the following as input:

- power distribution within the block
- interface boundary conditions from neighboring blocks
- block-specific geometry and material parameters

The solver outputs updated interface quantities (temperature or heat flux) after solving the local thermal problem.

In this work, AI-based local solvers are instantiated using a neural operator architecture. Specifically, we employ the U-NO [14] to approximate the mapping between block inputs and interface responses. The proposed framework is not tied to a specific model architecture, and other surrogate models (e.g., CNN-based models or physics-informed networks) can also serve as local block solvers within the same interface-iterative procedure.

E. Block Representation

To reduce modeling complexity while preserving dominant thermal behavior, each block is represented using a simplified

layered structure consisting of a substrate layer, a device layer, and an interconnect layer. The interconnect layer aggregates multiple back-end-of-line (BEOL) metal and via layers into a single effective thermal layer. This representation captures the dominant vertical thermal resistance while maintaining manageable input dimensionality for the learned solver.

F. Solver Interpretation

The proposed framework can be interpreted as a block-iterative linear solver for the discretized thermal system. After spatial discretization, the global thermal problem becomes

$$Ax = b \quad (5)$$

where $x \in \mathbb{R}^n$ collects the temperature unknowns across all blocks. Partitioning x by block $x = [x_1^\top, \dots, x_N^\top]^\top$ induces a block decomposition

$$A = D + R, \quad D = \text{blkdiag}(A_{11}, \dots, A_{NN}) \quad (6)$$

where each diagonal block A_{bb} captures internal thermal coupling within block Ω_b , and the off-diagonal remainder R is nonzero only at shared interface nodes. Isolating block b gives $A_{bb}x_b = b_b - \sum_{i \neq b} A_{bi}x_i$, showing that each block can be solved independently once its neighbor interface contributions are known.

The iterative update in (3) corresponds to a *relaxed block Jacobi* iteration [19]:

$$x^{(k)} = (1 - \omega)x^{(k-1)} + \omega D^{-1}(b - Rx^{(k-1)}) \quad (7)$$

Because D is block-diagonal, the inversion D^{-1} decouples into N independent local solves that can execute in parallel.

With exact numerical solvers for each block, this recovers a classical non-overlapping domain decomposition iteration [16]. When learned surrogates replace exact solves, the framework becomes an *inexact* block Jacobi iteration. Convergence is preserved provided the per-block surrogate error remains small relative to the interface residual [19].

III. EXPERIMENTAL RESULTS

A. Experimental Setup

Case Studies and Configurations. We evaluate DAIST on two representative 2.5D/3D-IC assemblies: the multi-chiplet system (Interposer–SoC–HBM) and the advanced packaging system (Interposer–MC–TIM), as depicted in Figure 2. In the multi-chiplet system, multiple SoC and HBM stacks are mounted on a shared interposer in a flip-chip assembly, with SoC and HBM dies attached face-down to the face-up interposer. To demonstrate composability, SoC instance counts are sampled uniformly from $\{1, 2, 3\}$ and HBM counts are sampled uniformly from $\{1, \dots, 10\}$. This sampling strategy produces assemblies with substantially diverse thermal footprints and interface coupling complexity. In the advanced packaging system, the molding compound (MC) is positioned face-down and covers the region of interest on the interposer, while the thermal interface material (TIM) is placed above the MC. Each block uses the three-layer abstraction from

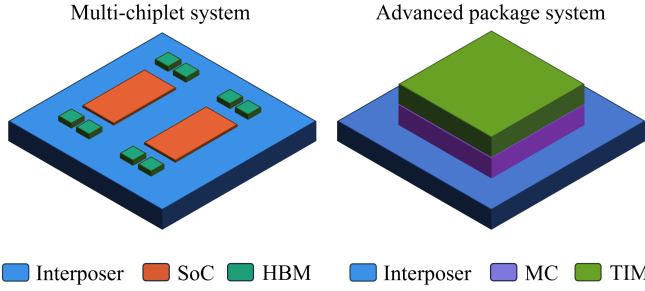


Fig. 2: Representative stackup schematics (not to scale) for the multi-chiplet assembly (Interposer–SoC–HBM, left) and the advanced packaging assembly (Interposer–MC–TIM, right): planar SoC and HBM placements on the shared interposer versus a vertical stack of molding compound (MC) and thermal interface material (TIM), consistent with the block parameters in Table I.

Section II (substrate, device, interconnect). Layer thicknesses, conductivities k , and volumetric heat generation follow Table I. Each layer follows a 50×50 in-plane discretization.

Boundary Conditions. Adiabatic (Neumann) conditions with zero normal heat flux are imposed on all exterior lateral faces. For both assemblies, top and bottom surfaces have a heat transfer coefficient (HTC) of $h = 100 \text{ W}/(\text{m}^2 \cdot \text{K})$, with the exception of the top surface of the interposer, which is adiabatic. For all surfaces with HTC, we use a fixed ambient temperature $T_\infty = 298.15 \text{ K}$. Internal block-to-block interfaces follow the iterative Dirichlet/Neumann exchange described in Section II without separate contact resistances.

Data Generation. For each block type (interposer, SoC, HBM, MC, TIM), a separate supervised dataset is generated with geometric parameters consistent with Table I. In the multi-chiplet system, power is generated exclusively in the SoC and HBM blocks, with the interposer carrying no internal heat sources. Power maps are sampled from Gaussian random fields (GRFs) with length-scale parameter 0.3. In the advanced packaging system, the interposer and TIM have no internal heat sources, and the MC block contains 1–4 embedded power regions at random locations. Each power region has a lateral dimension of $1 \text{ mm} \times 1 \text{ mm}$, with power profiles and interface boundary conditions sampled from the same GRF as described above. Each block dataset contains 5,000 total samples. For each sample, ground-truth fields are obtained from the Celsius Thermal Solver (abbreviated as Celsius). Footprints, layer thicknesses, conductivities, and external boundary conditions can all be treated as input variables for training. For simplicity, these variables are fixed in this work because variations in power, internal interface boundary conditions, and topology are the primary focus.

Training. For each block dataset, a ratio of 8:1:1 is used for splitting into training, validation, and test sets. This yields 4,000 training, 500 validation, and 500 test samples per block type. The interface relaxation parameters are set to $\omega_T = 0.5$ for temperature and $\omega_q = 0.5$ for heat flux, which are

selected via grid search on the validation set. Each model is trained for 1,000 epochs on 4 NVIDIA A100 GPUs, taking approximately 3 hours per block type. Training all five block types (interposer, SoC, HBM, MC, TIM) requires 15 hours in total, but these models are subsequently reused across any topology without retraining. By contrast, a monolithic predictor requires a full training run for each new package configuration, as its learned mapping is tied to a specific die count and arrangement.

Compared Methods. DAIST results were compared against the Celsius Thermal Solver and a monolithic neural predictor (ML baseline). Existing multi-block ML baselines for thermal simulation are limited. For example, single-block methods such as DeepOHeat-v1 [11] and PI-ONet [12] address fixed single-domain prediction and cannot extend to variable-topology multi-block assemblies without fundamental architectural changes and full retraining. Therefore, a monolithic U-NO predictor is adopted in this work as the ML baseline for multi-block thermal analysis. It is the most direct neural alternative to DAIST for full-package prediction, and it uses an identical architecture and hyperparameters to DAIST’s per-block models. A detailed description and analysis of this baseline is provided in Section III-C.

B. Multi-Chiplet System: Accuracy and Runtime

Figure 3 shows a three-dimensional temperature visualization of the multi-chiplet (Interposer–SoC–HBM) assembly with an example stackup of 2 SoC and 8 HBM blocks. DAIST’s prediction closely matches the Celsius reference across all blocks and layers. The interposer exhibits a thermal gradient from approximately 391 K at the periphery to 393.5 K beneath the high-power SoC footprints, with both methods capturing the same hotspot locations and magnitudes. The pointwise error field is within $\pm 0.5 \text{ K}$, with over 70% of pointwise errors below 0.2 K. The largest errors are localized at die-interposer interface regions where thermal gradients are steepest.

Table II summarizes DAIST’s accuracy and runtime on 100 held-out test cases from the multi-chiplet system, using the Celsius reference as the ground truth. DAIST achieves an MAE of 0.253 K, RMSE of 0.341 K, MAPE of 6.79×10^{-4} , and an average per-case maximum error of 1.135 K, while delivering a $178\times$ speedup over the full-package Celsius solve. A comprehensive comparison against the monolithic neural predictor, including both in-distribution and out-of-distribution evaluation, is presented in Section III-C.

Speedup. A full-package Celsius solve takes 146 s per case. DAIST’s per-iteration boundary update costs approximately 0.117 s on one NVIDIA A100 GPU. With a predefined stopping criterion of 0.1% relative change between consecutive iterations, DAIST requires on average 5.2 boundary iterations ($\approx 0.61 \text{ s}$) plus one final full-field reconstruction (0.091 s), for an end-to-end inference time of 0.82 s per case. This cost includes data-transfer and convergence-check overhead, and it is a speedup of $178\times$ over Celsius. Because each block

TABLE I: Block geometry and material parameters for Multi-chiplet System and Advanced Packaging System.

Block	Orientation	No. components	Footprint [‡]	Thickness (μm) [†]	k (W/m·K) [†]	Power (W) ^{†*}
<i>Case I: Multi-chiplet system (Interposer–SoC–HBM)</i>						
Interposer	face up	1	5×5 mm	(100, 1, 1)	(100, 2.175, 0.3)	(0, 0, 0)
SoC	face down	[1, 3]	2×1 mm	(10, 1, 1)	(100, 2.175, 0.3)	(0, [0.05, 0.1], 0)
HBM	face down	[1, 10]	0.5×0.4 mm	(20, 1, 1)	(100, 2.175, 0.3)	(0, [0.01, 0.03], 0)
<i>Case II: Advanced packaging system (Interposer–MC–TIM)</i>						
Interposer	face up	1	5×5 mm	(100, 1, 1)	(100, 2.175, 0.3)	(0, 0, 0)
MC	face down	1	3×3 mm	(100, 1, 1)	(100, 2.175, 0.3)	(0, [0.008, 0.08], 0)
TIM	face up	1	3×3 mm	(100, 1, 1)	(67, 67, 67)	(0, 0, 0)

[†]Values listed as (substrate layer, device layer, interconnect layer) triples.

*Power is listed as a range of values in [min, max] format.

[‡]Footprint is in-plane ($x \times y$).

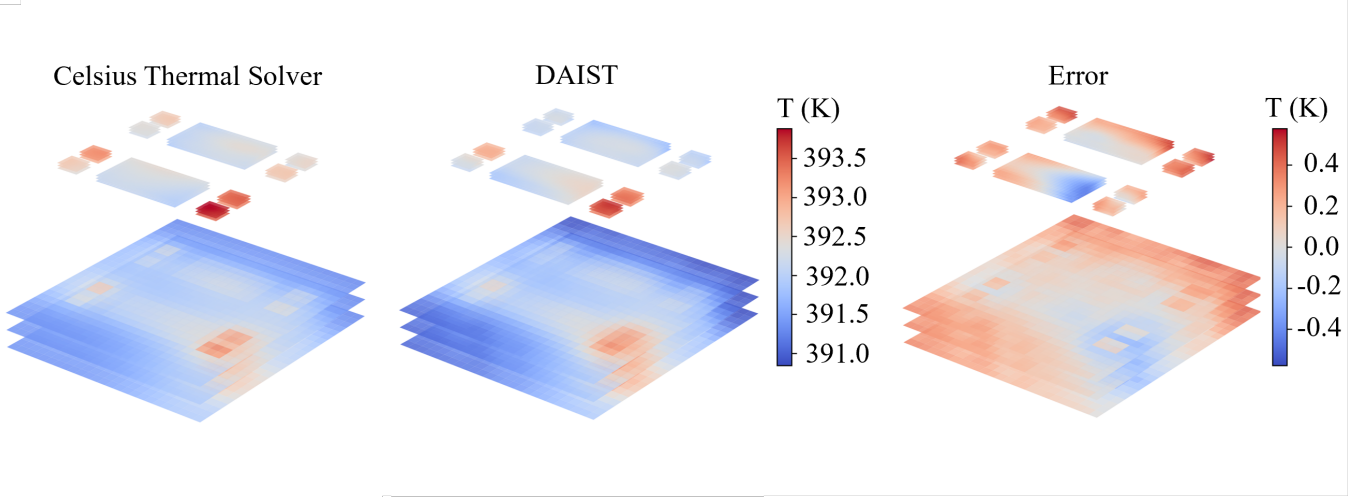


Fig. 3: Three-dimensional thermal visualization of the multi-chiplet assembly (Interposer–SoC–HBM). Left: Celsius Thermal Solver reference. Center: DAIST prediction. Right: pointwise absolute error.

TABLE II: Accuracy and runtime comparison on the multi-chiplet system.

Method	MAE (K)	Max (K)	Runtime (s)	Speedup
Celsius reference	—	—	146	$1\times$
DAIST	0.253	1.135	0.82	178$\times$

model is solved independently, all inferences of the same block type (interposer, SoC, HBM) within a single package are batched into one GPU call, making per-package latency largely independent of block count.

Accuracy–Runtime Tradeoff via Iteration Control. As discussed above, DAIST converges in 5.2 iterations on average under the default stopping criterion. However, users can manually control the tradeoff between accuracy and runtime. Table III shows the mean MAE on the 100-case test set when interface coupling is stopped after a fixed number of iterations (no early stopping). MAE decreases sharply through the first four iterations and saturates by iteration six. Table III extends to seven iterations to confirm this. Notably, even at the highest accuracy (seven iterations), the wall time remains well below

TABLE III: Mean MAE and wall time at fixed iteration counts for multi-chiplet test set.

Iterations	Mean MAE (K)	Wall time*
1	4.228	0.328
2	3.398	0.445
3	1.769	0.562
4	0.757	0.679
5	0.290	0.796
6	0.168	0.913
7	0.159	1.030
Celsius reference	—	146

*Includes field construction and other overhead time.

the 146 s Celsius reference. This allows users to control the iteration budget to balance runtime and precision requirements.

Figure 4 shows the convergence history of the interface residual $\delta^{(k)}$ for both case studies. Note that $\delta^{(k)}$ measures the change between consecutive iterations rather than the error against the ground-truth solution, so it decreases more steadily than MAE. The multi-chiplet system reaches the 0.1% tolerance in approximately 5 iterations, while the advanced

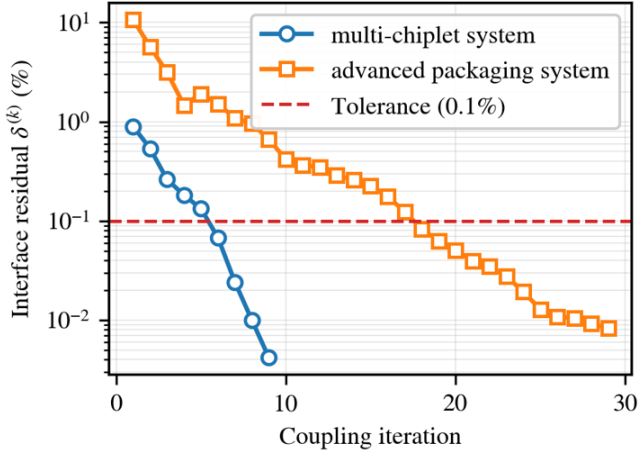


Fig. 4: Interface residual $\delta^{(k)}$ vs. coupling iteration for both case studies. The dashed line marks the convergence tolerance $\tau = 0.1\%$.

packaging system requires approximately 15 iterations. This is due to the difference in the coupling topology. In the multi-chiplet case, all dies share direct interfaces with the interposer, whereas the sequential chain (interposer $\rightarrow$ MC $\rightarrow$ TIM) requires multiple passes for thermal information to propagate end-to-end. The advanced packaging system is described in detail in Section III-D. Both cases exhibit stable and monotonic convergence, with appropriate tuning of the relaxation parameter ω .

C. Comparison with Monolithic Neural Predictor

DAIST is evaluated against the monolithic neural predictor, the most direct ML baseline for full-package thermal prediction, on the multi-chiplet system. This comparison focuses on whether compositional decomposition preserves in-distribution accuracy while enabling generalization to unseen topologies.

Setup. A fixed-topology dataset of 5,000 cases is constructed with the 2 SoC + 8 HBM configuration, and split by 8:1:1 into training, validation, and test sets. Locations and power maps of SoC and HBM blocks are randomized across all samples. Both DAIST and the monolithic predictor are trained using this dataset. This DAIST instance is trained under the same fixed-topology constraint as the monolithic predictor. It is distinct from the general DAIST evaluated in Section III-B, which is trained across variable topology counts. Accuracy metrics between the two subsections are therefore not directly comparable.

The monolithic predictor uses an identical U-NO architecture and hyperparameters to each per-block model in DAIST. Its inputs are the SoC power maps, HBM power maps, SoC footprint masks, HBM footprint masks, and external boundary conditions. Its output is the interposer top-surface temperature field. SoC and HBM masks are kept separate because they differ in height, power range, and thermal pathways. Notably, the monolithic predictor receives strictly richer input than any individual DAIST block model, including full power maps and

spatial masks for all dies. This gives the monolithic predictor an information advantage per forward pass. DAIST’s per-block models each receive only their local power map and interface boundary conditions, with no global topology information. This makes the comparison conservative for DAIST. If the monolithic predictor still fails on unseen topologies despite having access to more information, the failure is structural and cannot be attributed to any information asymmetry favoring DAIST.

Evaluation. Both methods are evaluated on two test sets of 100 cases each:

- **Fixed-topology:** held-out cases from the same 2 SoC + 8 HBM distribution (in-distribution).
- **Variable-topology:** cases drawn from the general multi-chiplet distribution in Section III-A, where SoC count is sampled uniformly from $\{1, 2, 3\}$ and HBM count from $\{1, \dots, 10\}$. This test set is out-of-distribution for *both* methods, as neither was retrained on variable topologies.

Although DAIST predicts temperature fields for all blocks, we report only the interposer top-surface temperature for fair comparison, as this is the only quantity both methods produce. Results are reported in Table IV.

In-Distribution Performance. On the fixed-topology test set, DAIST achieves lower MAE (0.282 K vs. 0.433 K) than the monolithic predictor. The monolithic predictor is faster at inference (0.12 s vs. 0.82 s) since it requires no iterative interface coupling. Both methods are accurate enough for practical use on the trained configuration, confirming that iterative decomposition does not sacrifice in-distribution accuracy. The monolithic predictor’s lower latency stems from the absence of iterative coupling overhead. Users with fixed topology and in-distribution operating conditions may prefer this increased speed in exchange for less flexibility. Conversely, DAIST’s iterative overhead is the necessary cost to achieve topology-agnostic generalization.

Out-of-Distribution Generalization. The critical distinction emerges on the variable-topology test set. DAIST’s MAE increases modestly from 0.282 K to 0.351 K (24% relative degradation). The monolithic predictor degrades catastrophically, with MAE rising from 0.433 K to 26.264 K, a $60\times$ increase showing its inability in generalization tasks.

This performance distinction arises from a fundamental structural difference. The monolithic predictor maps the power distributions of exactly 2 SoC and 8 HBM dies to a full-package temperature field. When the die count changes, it receives inputs of a different structure than it was trained on and fails to generalize. DAIST decomposes the problem along physical block boundaries, and each block model receives only its local power map and the interface boundary conditions from neighboring blocks. Neither the SoC nor the HBM model has information about how many sibling dies exist on the interposer, but rather they just respond to the flux or temperature at interfaces. The interposer model likewise receives flux contributions from whatever blocks contact it, regardless of their count or arrangement. This localized phys-

TABLE IV: Accuracy and runtime comparison of DAIST and the monolithic neural predictor on fixed-topology and variable-topology test sets.[†]

Method	Test Set	Runtime (s)	MAE (K)	Max (K)	MAPE	RMSE (K)
DAIST	Fixed-topology	0.82	0.282	1.230	7.01×10^{-4}	0.356
	Variable-topology		0.351	1.233	9.10×10^{-4}	0.415
Monolithic	Fixed-topology	0.12	0.433	0.937	1.07×10^{-3}	0.474
	Variable-topology		26.264	26.944	6.83×10^{-2}	26.269

[†]MAE: mean absolute error; Max: mean of per-case maximum absolute pointwise error; MAPE: mean absolute percentage error relative to ground truth temperature; Runtime: wall-clock inference time per case.

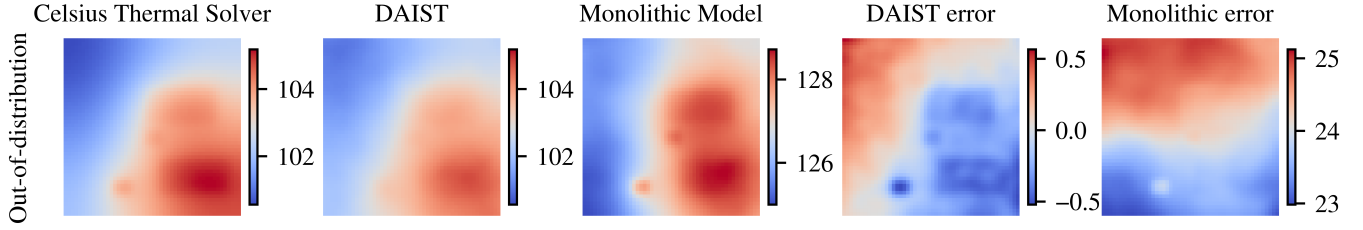


Fig. 5: Out-of-distribution comparison on a variable-topology case. Left to right: Celsius Thermal Solver reference, DAIST prediction, monolithic prediction, DAIST error, monolithic error.

ical representation enables DAIST to generalize to topologies never encountered during training.

DAIST’s modest OOD degradation (0.282 K to 0.351 K) shows extrapolation ability at the individual block-level. In unseen topologies, the interposer receives interface flux patterns outside the training distribution. Nevertheless, the interposer model generalizes well because the underlying thermal response, governed by the heat equation, maintains a consistent input–output structure. Locally, higher heat flux leads to proportionally higher temperatures, and globally, the aggregated heat flux remains linearly related to the mean temperature rise through an effective thermal resistance. Since these relationships are independent of die count or placement, the learned operator can extrapolate to unseen flux patterns without relying on topology-specific features.

Figure 5 visualizes this contrast on a representative variable-topology case. The Celsius reference and DAIST prediction show close agreement in both temperature magnitude and hotspot location, with DAIST errors within ± 0.5 K across most of the field. The monolithic predictor captures the general spatial pattern of hotspot locations, which is expected since the correlation between local power input and temperature rise is straightforward to learn. However, it overestimates the absolute temperature by approximately 24 K, indicating that the model fails to correctly resolve the effective thermal resistance of the unseen topology. This confirms that the monolithic predictor’s failure is not in spatial feature extraction but in its inability to generalize the global thermal balance to configurations outside the training distribution.

Beyond accuracy, DAIST’s compositional structure also reduces training overhead. When a new topology is introduced, only novel block types need to be trained, while existing block models are frozen and reused at zero additional cost.

TABLE V: Cross-topology block reuse: training and testing configurations. The interposer model is frozen from the multi-chiplet system and receives no MC–TIM training data.

Block	Trained on	Tested in
Interposer	SoC–HBM (frozen)	MC–TIM
MC	MC–TIM	MC–TIM
TIM	MC–TIM	MC–TIM

D. Cross-Topology Validation: Interposer–MC–TIM

As established in Section III-A, the interposer block carries no internal heat sources in either system configuration, meaning it learns only conduction-driven thermal transport rather than power-driven transport. This property motivates a direct test of compositional reusability. A DAIST instance for the advanced packaging system (Interposer–MC–TIM) is assembled using the interposer block model trained exclusively on multi-chiplet (Interposer–SoC–HBM) data. The interposer block model stays frozen without any fine-tuning. The MC and TIM block models are each trained independently on their own separately generated 5,000-case datasets with an 8-1-1 split. Table V summarizes the training and testing configuration for each block, confirming that the interposer model is evaluated on a stack configuration unseen during training.

Figure 6 shows the three-dimensional temperature field for a representative test case of the advanced packaging system. The Celsius reference exhibits two hotspots near the left corner, with the interposer layer alone accounting for a temperature range of approximately 2 K. DAIST captures the same spatial hotspot pattern but with two notable discrepancies. First, DAIST shows a compressed thermal gradient (under 1 K vs. 2 K). Second, the ground truth exhibits a sharp temperature

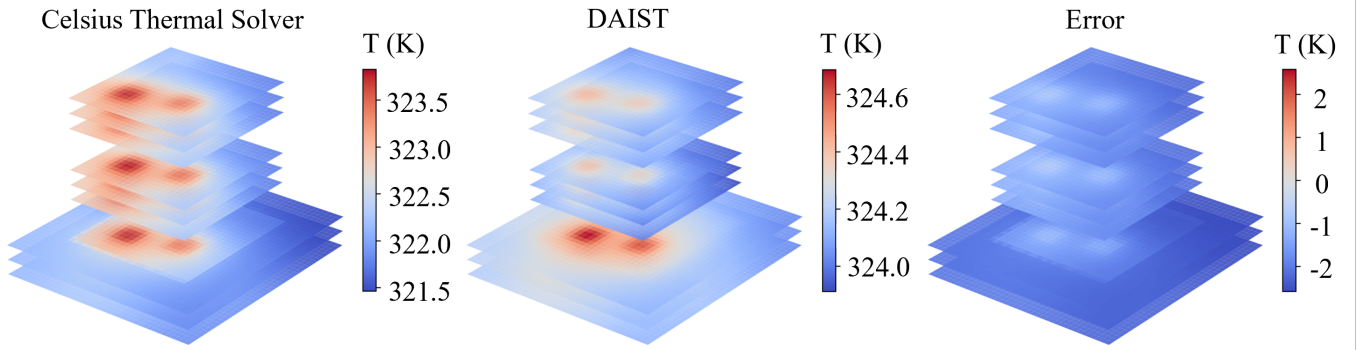


Fig. 6: Three-dimensional thermal visualization of the advanced packaging assembly (Interposer–MC–TIM) with a frozen interposer model transferred from the multi-chiplet system. Left: Celsius Thermal Solver reference. Center: DAIST prediction. Right: pointwise error.

discontinuity at the interposer–MC interface between the MC and exposed regions, which DAIST fails to reproduce. Both discrepancies are concentrated in the interposer layer, with errors reaching up to ± 2 K.

These discrepancies are expected. In the multi-chiplet system, the interposer receives heat from multiple discrete die footprints with independently sampled power maps, producing a distributed thermal load across its surface. By contrast, the advanced packaging system concentrates heat into 1–4 small regions within a single MC block, creating a much sharper, more distinct flux pattern. The frozen interposer block has never encountered this flux distribution, which accounts for both the compressed thermal gradient and the missing temperature discontinuity at the MC boundary. Closing this domain gap through lightweight fine-tuning on a small number of advanced packaging samples is a natural direction for future work.

Validated on 100 held-out MC–TIM test cases, DAIST achieves an MAE of 1.056 K, RMSE of 1.135 K, MAPE of 3.23×10^{-3} , and a mean per-case maximum absolute error of 2.396 K. These cases were entirely unseen by the interposer model during training. While accuracy decreases relative to the in-distribution multi-chiplet result (0.253 K MAE), this reflects the cost of zero-retraining cross-topology reuse. Specifically, the interposer model generalizes to a fundamentally different stack (from multiple chiplets to a single MC) without any exposure to MC–TIM training data.

Convergence and Speedup. With a predefined convergence criterion of 0.1%, DAIST requires on average 15.3 interface iterations to converge for the three-block sequential chain (interposer $\rightarrow$ MC $\rightarrow$ TIM). This higher iteration count relative to the multi-chiplet case (5.2 iterations) is due to the sequential topology of the assembly. The interposer has no direct interface with the TIM block, so thermal information must propagate through an intermediate MC block before the full chain reaches consistency. In a parallel coupling graph (as in the multi-chiplet system, where all dies interface directly with the interposer), interface inconsistencies are corrected in fewer global passes. However, in a linear chain, each pass can

TABLE VI: Mean MAE and wall time at fixed iteration counts for advanced packaging test set.

Iterations	Mean MAE (K)	Wall time (s)
1	33.282	0.328
5	5.657	0.796
10	1.914	1.381
15	1.239	1.966
20	0.997	2.551
Celsius reference	—	197

only propagate information one hop further, requiring more iterations. Table VI shows the full iteration-wise breakdown. The reference solve for the full advanced packaging system takes 197 s. DAIST converges in approximately 2 s, yielding a speedup of $99\times$.

IV. CONCLUSION

We presented DAIST, a composable thermal solver that decomposes heterogeneous 2.5D/3D-IC packages into modular blocks, replaces subdomain solvers with neural operators, and couples them through iterative interface exchange of temperature and heat flux. Evaluated on a multi-chiplet system and an advanced packaging system, DAIST achieves up to $178\times$ speedup over Celsius with mean temperature errors of 0.068% and 0.323%, respectively. Notably, the advanced packaging result is achieved with a frozen interposer model transferred directly from the multi-chiplet system, demonstrating cross-topology reuse of block-level models without retraining. Future work will explore transfer learning to narrow the cross-topology accuracy gap, extend the framework to more complex 3D-IC packages, and apply DAIST to thermal-aware design optimization.

REFERENCES

- [1] J. H. Lau, “Recent advances and trends in advanced packaging,” *IEEE Transactions on Components, Packaging and Manufacturing Technology*, vol. 12, no. 2, pp. 228–252, 2022.

- [2] V. Lin, S.-Y. Hou, T.-H. Huang, C.-H. Hsieh, S. Lu, S. Guo *et al.*, “Wafer-level integration of an advanced logic-memory system through the second-generation CoWoS technology,” *IEEE Transactions on Electron Devices*, vol. 64, no. 10, pp. 4071–4077, 2017.
- [3] P. Li, L. T. Pileggi, M. Asheghi, and R. Chandra, “Efficient full-chip thermal modeling and analysis,” in *IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, 2004, pp. 319–326.
- [4] Y.-C. Chen, S. Ladenheim, C. Kalargaris, M. Mihajlović, and V. F. Pavlidis, “Computationally efficient standard-cell FEM-based thermal analysis,” in *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2017, pp. 490–495.
- [5] A. Sridhar, A. Vincenzi, D. Atienza, and T. Brunschweiler, “3D-ICE: A compact thermal model for early-stage design of liquid-cooled ICs,” *IEEE Transactions on Computers*, vol. 63, no. 10, pp. 2576–2589, 2014.
- [6] H. Sultan, A. Chauhan, and S. R. Sarangi, “A survey of chip-level thermal simulators,” *ACM Computing Surveys*, vol. 52, no. 2, pp. 1–35, 2019.
- [7] J.-M. Lin, W.-Y. Chang, H.-Y. Hsieh, Y.-T. Shyu, Y.-J. Chang, and J.-M. Lu, “Thermal-aware floorplanning and TSV-planning for mixed-type modules in a fixed-outline 3-D IC,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 29, no. 9, pp. 1652–1664, 2021.
- [8] L. Lu, J. Zhu, Y. Li, A. Nagarajan, J. Liu, S.-Y. Shiau, and X. Ai, “3D-IC in-design thermal analysis and optimization,” in *Proc. IEEE Intersociety Conference on Thermal and Thermomechanical Phenomena in Electronic Systems (ITherm)*, 2024.
- [9] M. Raissi, P. Perdikaris, and G. E. Karniadakis, “Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations,” *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019.
- [10] Z. Liu, Y. Li, J. Hu, X. Yu, S. Shiau, X. Ai, Z. Zeng, and Z. Zhang, “DeepOHeat: Operator learning-based ultra-fast thermal simulation in 3D-IC design,” in *Proceedings of the 60th ACM/IEEE Design Automation Conference (DAC)*, 2023, pp. 1–6.
- [11] X. Yu, Z. Liu, H. Li, Y. Li, X. Ai, Z. Zeng, I. Young, and Z. Zhang, “DeepOHeat-v1: Efficient operator learning for fast and trustworthy thermal simulation and optimization in 3D-IC design,” *arXiv preprint arXiv:2504.03955*, 2025.
- [12] Y. Sha, C. Zhang, and Q. Chen, “PI-ONet: A physics-informed operator network for efficient thermal analysis of multilayer chiplets,” *IEEE Transactions on Components, Packaging and Manufacturing Technology*, 2025.
- [13] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar, “Fourier neural operator for parametric partial differential equations,” in *International Conference on Learning Representations (ICLR)*, 2021. [Online]. Available: <https://openreview.net/forum?id=c8P9NQVtmnO>
- [14] M. A. Rahman, Z. E. Ross, and K. Azizzadenesheli, “U-NO: U-shaped neural operators,” *Transactions on Machine Learning Research*, 2023. [Online]. Available: <https://openreview.net/forum?id=j3oQF9coJd>
- [15] Z. Huang, H. Wang, W. Yang, M. Tang, D. Xie, T.-J. Lin, Y. Zhang, W. W. Xing, and L. He, “Self-attention to operator learning-based 3D-IC thermal simulation,” in *Proceedings of the 62nd ACM/IEEE Design Automation Conference (DAC)*, 2025, arXiv:2510.15968.
- [16] Y. Yang, C.-K. Zhan, J. Gu, and P. Li, “Fast 3-D thermal simulation for integrated circuits with domain decomposition method,” in *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2014, pp. 1–6.
- [17] C. Mao, R. Lupoiu, T. Dai, M. Chen, and J. A. Fan, “Towards general neural surrogate solvers with specialized neural accelerators,” in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 235. PMLR, 2024, pp. 34 693–34 711.
- [18] R. Wu, N. Kovachki, and B. Liu, “A learning-based domain decomposition method,” *arXiv preprint arXiv:2507.17328*, 2025.
- [19] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed. SIAM, 2003.